

BACK ENDS

Flask and Dynamic Templating

Frameworks: Packaged tools and techniques that aid in the creation of web apps.

Virtual environment: A simulated environment that runs in isolation from the rest of a machine.

Application server: A server that handles HTTP requests and responses, and also provides business logic to the client.

Flask: A Python-based microframework that's relatively simple and lightweight.

Sample code to return an HTML file in Flask:

```
from flask import Flask, render_template
app = Flask(__name__)
@app.route("/", methods=['GET'])
def welcome():
    return render_template("home.html")
```

Jinja2: Software that Flask applications use to customize certain parts of their HTML templates.

Sample code to pass data to an HTML file:

Python file:

```
def inventory():
    return render_template("inv.html", beans = 5)
```

HTML file:

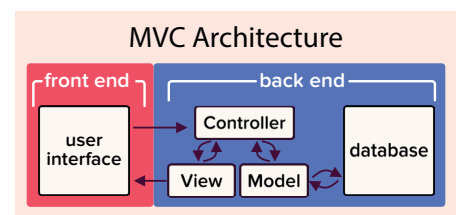
```
<p>{{beans}} bags of beans left!</p>
```

Architecture: The structure that defines how a web app's code and components should interact.

Model-View-Controller (MVC): A common architecture pattern that separates a web app's user interface from its business logic.

- The Model communicates with the database.
- The View builds the user interface.
- The Controller handles HTTP requests, which includes returning HTTP responses, and it acts as a broker between the Model and the View.

In Flask, HTML files are considered templates. CSS files, JS files, and images are considered static assets.



Forms, Error Handling, and Database Integration

Request object: A format for storing code from a client request that can be accessed on the back end via attributes.

Error handling: Checks that data inputs meet certain requirements and notifies the user when any of those requirements aren't met.

Cross-Site Request Forgery (CSRF): A common attack in which a fake form mimics a **POST** request and tricks the server into giving the attacker access to private data.

CSRF token: A long string of random characters generated by a server and attached to a form. If a **POST** request contains the token, the server can validate the request.

WTForms: A library that provides tools for building and maintaining secure web forms.

Object-relational mapping (ORM): Provides an object-based framework on top of the database.

ORM enables devs to call methods on database tables and records.

REST API

Representational State Transfer (REST): A set of six key principles that standardize how servers should make data available and how that data should be transferred to a client.

1. **Client-server architecture:** The front end and back end are separate.
2. **Uniform interface:** A logical and standard way of fetching back-end resources that's the same no matter how the front end is configured.
3. **Stateless:** The server shouldn't store data or remember client sessions.
4. **Caching:** A process that allows a web browser to temporarily store files on the user's hard drive to reduce the number of interactions between the client and the server.
5. **Layered system:** It should be possible to deploy the back end across several layers of servers without impacting the front end.

Decoupling the front end and back end allows dev teams to:

- Hire specialized engineers
- Split into two smaller teams
- Optimize the front and back ends separately
- Mix and match frameworks and libraries

CRUD stands for Create, Read, Update, and Delete: four basic operations that roughly describe HTTP communication between the front end and back end.

JavaScript Object Notation (JSON): A lightweight and easy-to-implement format for storing and transporting data.

Postman: A popular online tool that simulates front-end HTTP requests to aid back-end testing.

`get_json()` is a Flask method that extracts the JSON from a request object.

`jsonify()` is a Flask function that can turn Python code into a JSON response object.

Integrating ReactJS

ReactJS: A commonly-used, open-source JavaScript library.

React hooks: Functions that provide state and other features without requiring the creation of a class. Two common hooks are:

- The `useState` hook, which prepares a state to store API data
- The `useEffect` hook, which manages the request for data from the API

Promise: A JavaScript object which stands in for pending values.

Handlers: Methods of a promise object that execute code when the promise's status changes from pending to fulfilled or rejected.

Deployment

Content delivery network (CDN): A PaaS solution for deploying a static front end, which replicates content across a geographically distributed network using caching.